BTK AKADEMİ

4. GÜN | LLM API + OLLAMA BOOTCAMP

01 Kasım 2026 · Kalan kazanımlar + zaman kalırsa 5. güne giriş

Python → API → Ollama → JSON → Doğrulama

Nereden devam ediyoruz?

4. günün işlenmeyen dört kazanımı

01 · LLM API

İstek–yanıt mantığı, HTTP ve güvenlik ilkeleri

02 · Python çağrısı

requests ile ilk POST ve yanıtı okuma

03 · Yerel model

Ollama kurulumu + Python ile model kullanımı

04 · Ayarlar ve yöntem

Ollama üretim ayarları + yöntemlerin farkı

Önceden işlenen refactoring ve unit test tekrar dersine dönmüyoruz.

Bugün çalışan çıktı ne?

Bir destek kaydı Python programına girilecek.

Python › **HTTP isteği** › **Ollama** › **LLM yanıtı** › **JSON kontrolü**

Kodu yazmak değil, çıktıyı kontrol etmek başarı ölçütü.

Zamanımızı nasıl kullanacağız?

İlk 260 dakika 4. gün; kalan 40 dakika şartlı 5. gün başlangıcı.

30 dk

Ollama + modelin çalışması

60 dk

API mantığı + Python HTTP çağrısı

90 dk

JSON + hata yönetimi + test + güvenlik

80 dk

Ollama ayar lab + yöntem seçimi

4. gün tamamlanmadan opsiyonel 5. gün bölümüne geçme.

Terminalde Ollama’yı başlat

Önce yerel modelle gerçekten konuş.

```
ollama --version
ollama list
ollama pull qwen2.5:1.5b
ollama run qwen2.5:1.5b
```

İndirme yaklaşık 986 MB; donanımı yetersiz öğrenciler eşleşsin.

Kurulum takılırsa ders durmayacak

Ollama için üç kontrol

Model yok

ollama list; gerekiyorsa pull komutu

Servis erişilemiyor

Masaüstü Ollama açık mı? Gerekirse ollama serve

Yavaş veya RAM yetmiyor

Daha küçük model veya eşli çalışma

Yerel modelin varlığı, iyi yanıt verdiğini garanti etmez.

Yerel LLM gerçekten ne demek?

Veri yolu ile kullanım yetkisini birbirinden ayır.

Yerel çıkarım

Model bilgisayarda hesaplama yapar; hız donanımla sınırlıdır.

Bulut API

Servise ağ üzerinden gidilir; anahtar, maliyet ve veri politikası önemlidir.

Güvenlik sınırı

Yerel HTTP API varsayılan olarak kimlik doğrulamasızdır; internete açılmaz.

“Yerel” = otomatik olarak güvenli / ucuz / doğru değildir.

LLM API isteği nasıl çalışır?

İlk kez gerçek bir HTTP istek–yanıt döngüsü kuruyoruz.



200 OK ≠ teknik olarak doğru sınıflandırma.

API güvenliğinin 3 temel kuralı

Önce uygulamaya güvenlik alışkanlığı kazandır.

Girdi

Gerçek müşteri bilgisi, parola veya kişisel veri kullanma.

Kimlik bilgisi

Bulut anahtarını koda ve Git deposuna yazma.

Çıktı

LLM yanıtını komut veya kesin karar gibi uygulama; doğrula.

Sadece model yanıtı değil, erişim ve veri akışı da kontrol edilmeli.

Uygulama 1 · GET /api/tags

Öğrenciler 01_servis_kontrol.py dosyasını tamamlasın.

```
import requests
URL = "http://localhost:11434/api/tags"
response = requests.get(URL, timeout=5)
response.raise_for_status()
models = response.json()["models"]
print([m["name"] for m in models])
```

TODO-1: GET → HTTP durumu → JSON alanı → model listesi

Uygulama 2 · İlk Python → Ollama çağrısı

02_ilk_api_cagrisi.py dosyasındaki POST işlemini yaz.

```
import requests
URL = "http://localhost:11434/api/chat"
payload = {"model": "qwen2.5:1.5b",
           "messages": [{"role": "user", "content": "Merhaba"}],
           "stream": False}
response = requests.post(URL, json=payload, timeout=180)
response.raise_for_status()
print(response.json()["message"]["content"])
```

POST /api/chat + stream:false → tek JSON yanıtı

İstek bozulursa ne olur?

Üç ayrı hata deneyini sırayla uygula.

Ollama kapalı

Bağlantı hatası: sunucudan HTTP cevabı gelmez.

Yanlış model adı

HTTP hata durumu ve açıklaması gelebilir.

Çok kısa timeout

İstek süresi aşılır; bu modelin yanlış yanıtı değildir.

`try/except` ve `raise_for_status()` farklı hataları yakalar.

JSON iki ayrı katmandadır

Dış servis yanıtıyla iç model çıktısını ayır.

1 · API yanıtı

`response.json()` → status/model/message gibi alanlar

2 · Model metni

`response.json()["message"]["content"]` → metin

3 · JSON parse

`json.loads(content)` → Python dict; yine doğrulama gerekir

Geçerli JSON, doğru veya yetkili işlem anlamına gelmez.

Uygulama 3 · Destek asistanı

03_destek_asistani.py: prompt → format:json → yanıt.

```
Kategoriler: Erişim / Performans
             Veri Kalitesi / Diğer
Alanlar: kategori, oncelik, gerekce,
         eksik_bilgiler
```

Öğrenci, yalnızca bir cevabı değil veri sözleşmesini üretecek.

Uygulama 4 · Doğrulama fonksiyonu

LLM ne döndürürse döndürsün Python iş kurallarına uymalı.

Alan denetimi

kategori, oncelik, gerekce, eksik_bilgiler eksiksiz

Değer denetimi

Kategoriler ve öncelikler izinli değer kümesinde

Tip denetimi

Gerekçe string, eksik_bilgiler string listesi

TODO-3: dogrula() — eksik/yanlış veri → ValueError

Üç gerçekçi destek senaryosu

Öğrenciler aynı kodu üç kayıtla çalıştırsın.

T-001 · Performans

Finans raporu yavaş; 60 kullanıcı etkileniyor.

T-002 · Erişim

Parola değişti, kullanıcı giriş yapamıyor.

T-003 · Veri kalitesi

Satış toplamları muhasebe verisiyle uyuşmuyor.

Model önceliğini kurumsal SLA olmadan kesin karar sayma.

Zor vakalar: Program ne yapmalı?

Soruların yanıtını kod ile gösterin.

Geçersiz JSON

json.loads() hata verir → kayıt hata mesajıyla reddedilir.

Geçerli ama yanlış şema

dogrula() hata verir → otomatik işlem yok.

Yanlış ama iyi biçimlenmiş karar

İnsan/senaryo testi gerekir; şema doğrulaması yetmez.

“JSON doğrulandı” demek “sorun çözüldü” demek değildir.

04_testler.py · Birim test çalıştır

Modeli çalıştırmadan Python iş kurallarını test et.

```
python -m unittest -v 04_testler.py
test_gecersiz_kategori
test_eksik_alan
test_olmayan_servis_hatasi
```

Testler eksik TODO fonksiyonları tamamlanınca yeşile dönmeli.

Ollama model ayarları: Neyi değiştiriyoruz?

Aynı model ve aynı soru; üretim davranışını değiştiren kontroller.

Model ağırlıkları

Eğitim sonunda öğrenilmiş parametreler. Bugünkü atölyede bunları değiştirmiyoruz.

İstem ve bağlam

Kullanıcı talimatı, rol, belge ve sohbet geçmişi. Modelin gördüğü girdiler.

Çıkarım ayarları

Token seçimi, bağlam kapasitesi ve çıktı sınırı. options alanıyla deney yapacağız.

Temperature değiştirmek = modeli yeniden eğitmek değildir.

Ayarları değiştirebileceğimiz 3 yer

Terminalde hızlı deneme; Python API’de istek bazlı kontrol; Modelfile ile kalıcı profil.

Terminal oturumu

ollama run qwen2.5:1.5b
/set parameter temperature 0.2
Bu oturumda dene.

Python /api/chat

"options": {
  "temperature": 0.2,
  "num_ctx": 4096
}
Her çağrıya ayrı ayar.

Modelfile

FROM qwen2.5:1.5b
PARAMETER temperature 0.2
SYSTEM "..."
Kendi profilini oluştur.

Bugün önce terminal, ardından Python options ve en son Modelfile.

Terminal laboratuvarı · Bir ayarı değiştir

Önce tahmin et, sonra terminalde aynı soruyu iki ayarda dene.

```
ollama run qwen2.5:1.5b

/set parameter temperature 0.1
Finans raporu yavaşsa ilk 3 kontrol nedir?

/set parameter temperature 1.0
Finans raporu yavaşsa ilk 3 kontrol nedir?
```

Soru sabit, model sabit, yalnız temperature değişsin.

Temperature: Rastgelelik ve doğruluk aynı şey değil

Üç ayarı karşılaştır; cevapların faydasını ölçmeden “iyi” deme.

0.1 · Düşük

Tekrarlı işlerde daha kararlı çıktı arayışı. Sınıflandırma ve yapılandırılmış cevap deneyi.

0.7 · Orta

İfade çeşitliliği ile odak arasında başlangıç dengesi. Kurum iş kuralları yine gerekli.

1.0 · Yüksek

Daha çeşitli ve bazen daha sürpriz ifadeler. Fikir üretimi için denenebilir; doğruluk ölçülmeli.

Daha düşük sıcaklık = genellikle daha tutarlı, ama otomatik doğru değil.

Top-k, top-p ve tekrar cezası

Üçü farklı kontrol; hepsini aynı anda değiştirirsen neyin etkilediğini anlayamazsın.

top_k

Token seçiminde en olası K adayın dışını eler.
Örnek: 10 ile 80'i karşılaştır.

top_p

Kümülatif olasılık toplamı p sınırına ulaşan aday kümesi.
Örnek: 0.4 ve 0.9.

repeat_penalty

Yakın dönemde tekrar edilen tokenlara ceza uygular.
Örnek: 1.0 ile 1.2.

Bir deneyde yalnızca tek parametreyi değiştir.

Context ve çıktı uzunluğu: iki ayrı sınır

num_ctx modelin aynı anda işleyebildiği bağlamı; num_predict çıktı token sınırını ayarlar.

num_ctx = 4096

İstem + sohbet geçmişi + belge parçaları için çalışma alanı. Desteklenen kapasite modele ve belleğe bağlı.

num_predict = 40

Modelin üreteceği maksimum yeni token sayısını sınırlar. 40 token uzun açıklamayı kesebilir.

Gözlem / ölçüm

num_predict 40 ve 180 ile aynı istemi dene. Tamamlanan gerekçe ve eksik alanları karşılaştır.

num_ctx artışı bellek kullanımını yükseltebilir; num_predict kısa kalırsa yanıt kesilir.

Seed, stream ve keep_alive: Farklı amaçlar

Bazıları token seçimi, bazıları iletim biçimi ve servis kaynak kullanımıyla ilgilidir.

seed = 42

Aynı deneyin tekrarını kolaylaştırmak için rastgelelik başlangıcı.
Model/sürüm/donanım da etkili olabilir.

stream = false

Tek HTTP cevabı al; dersimizde JSON parse kolay olsun. True olursa akış parçaları gelir.

keep_alive = "5m"

Modelin isteğin ardından bellekte ne kadar tutulacağını yönetir. Yanıt kalitesini doğrudan artırmaz.

Seed tekrar denemeyi kolaylaştırır; tüm ortamlarda birebir aynı cevap garantisi değildir.

Python API’de ayarların yeri

Aynı model, aynı prompt, farklı options nesnesi.

```
import requests
URL = "http://localhost:11434/api/chat"
soru = "Finans raporu yavas aciliyor."
payload = {
  "model": "qwen2.5:1.5b",
  "messages": [{"role": "user", "content": soru}],
  "stream": False,
  "options": {"temperature": 0.2, "top_p": 0.9,
            "seed": 42, "num_predict": 160}
}
response = requests.post(URL, json=payload, timeout=180)
response.raise_for_status()
print(response.json()["message"]["content"])
```

Öğrenciler 08_ayar_deneyi.py içindeki TODO-A’yı tamamlasın.

Uygulama A · Parametre karşılaştırma deneyi

Tek değişken değiştir; sonucu ölçmeden “daha iyi” deme.

Sabit koşullar

Aynı model, aynı Türkçe destek istemi, aynı donanım ve seed. Diğer parametreler aynı.

Deney profilleri

A: temp 0.1
B: temp 1.0
C: top_p 0.35
D: num_predict 40

Ölç ve tartış

Yanıt tam mı?
Kanıtsız iddia var mı?
Tekrar var mı?
İşlem süresi ne kadar?

En az 3 profili çalıştır; beklenen alanları ve süreyi kaydet.

Uygulama B · JSON formatı ≠ iş kuralı

Ayarlarla yapılan üretimi geçerli iş sonuçlarıyla karşılaştır.

1 · JSON format

format="json" veya JSON Schema iste. Yanıt JSON olarak ayrıştırılmalı.

2 · Tip / alan kontrolü

kategori, oncelik, gerekce gibi alanlar var mı? Doğru veri tiplerinde mi?

3 · İş kuralı kontrolü

Kategori yalnızca tanımlı dört etiketten biri mi? Öncelik gerekçesi kanıtlı mı?

Format ve içerik iki ayrı kapıdan geçmeli: parse → iş kuralı doğrulaması.

Modelfile · Kurumsal profil oluşturalım

Aynı modeli tekrar tekrar aynı sistem mesajı ve ayarlarla başlat.

```
# Modelfile.destek
FROM qwen2.5:1.5b
PARAMETER temperature 0.2
PARAMETER top_p 0.9
PARAMETER num_ctx 4096
PARAMETER num_predict 180
SYSTEM """
Türkçe yanıt ver. Kanıtsız kesin teşhis koyma.
Eksik bilgileri açıkça belirt.
"""


# Terminal: ollama create btk-destek -f Modelfile.destek
# Terminal: ollama run btk-destek
```

ollama create eğitim değildir; mevcut model için yapılandırılmış bir çalıştırma profili oluşturur.

Ayar laboratuvarı · Son kontrol

4. günün eksik kazanımlarını bitir; yalnız herkes başardıysa sonraki konulara geç.

Çalışan kod

Python → /api/chat. Aynı istemi iki seçenekle çağır ve yanıtı ekrana yaz.

Kontrollü deney

Temperature/top_p ve num_predict farkını tek değişkenli deneyle yorumla.

Güvenli sonuç

JSON doğrulaması, hata kontrolü, yetki ve gizlilik önlemleri ayarlardan bağımsız.

Başarı ölçüsü: ayarı değiştirmek + etkisini gözlemek + güvenilirlik sınırını açıklamak.

Güvenlik mini tatbikatı

Gerçek veriler kullanmadan riskleri sorgula.

Sızıntı

Koda bir bulut API anahtarı gömülürse ne olur?

Kontrolsüz karar

Model “Kritik” dedi diye servis yeniden başlatılır mı?

İzlenebilirlik

Hangi kayıt, hangi model, hangi sonuç? Hassas veriyi loglama.

İnsan onayı ve erişim kontrolü promptla ikame edilmez.

Prompt, RAG, Fine-Tuning: hangisi?

Sadece ihtiyaç üzerinden karşılaştır.

Prompt Engineering

Görev, bağlam, kısıt ve çıktı biçimini yönlendirir.

RAG

Güncel kurumsal dokümanı dışarıdan getirip bağlama ekler.

Fine-Tuning

Model davranışını örneklerle ağırlık güncelleyerek uyarlar.

Teknikler rakip değil; gerektiğinde birlikte kullanılır.

4. gün bitiş kontrolü

Servis, Python çağrısı ve ayar deneyini göster.

Çalışan servis

Ollama modeli terminalden çalışıyor.

Çalışan Python

POST /api/chat ile yanıt geliyor.

Kontrol edilen çıktı

JSON + parametre karşılaştırması ve hata kontrolleri yapılıyor.

Üçü de sağlandıysa 5. günün giriş bölümünü aç.

OPSİYON · 5. güne neden geçiyoruz?

4. günün API, JSON ve ayar deneyleri tamamlandıysa aç.

LLM sorunu

Kuruma ait güncel prosedürü eğitim sırasında öğrenmemiş olabilir.

RAG yaklaşımı

İlgili dokümanı önce bul, sonra bağlama ekle.

Bugünün sınırı

Yalnız belge hazırlama, chunking ve metadata; embedding sonraki blokta.

5. günün ilk kazanımına geçiş — zorunlu değil.

Opsiyon · Kurumsal dokümanları Python ile oku

06_opsiyonel_dokuman_ve_chunking.py

```
belgeler = belgeleri_oku()
print(len(belgeler))
for doc in belgeler:
    print(doc["doc_id"], doc["team"])
```

Kurgusal beş belge: Superset, PostgreSQL, erişim, rapor, toplantı.

Opsiyon · Chunking ve overlap

Bir dokümanı arama için parçalara ayırıyoruz.

Chunk boyutu

Çok büyük: ilgisiz bilgi; çok küçük: bağlam kaybı.

Overlap

Komşu parçaların sınırlarında bir miktar tekrar.

Öğrenci görevi

chunk_words(text, size=20, overlap=5)

Kelime bazlı oyuncak örnek; gerçek sistemler token/bölüm sınırlarını da dikkate alır.

Opsiyon · Metadata ve izlenebilirlik

Parça içeriği tek başına yeterli değil.

chunk_id

Kaynak parçanın tekil kimliği.

doc_id ve team

Belge kaynağı ve ilgili ekip.

visibility

Yetki kontrolüne yardımcı etiket; tek başına yetki sistemi değil.

Her parçaya kaynak ve sıra bilgisi ekle.

Opsiyon · Gün sonu kontrolü

Gelecek dersin retrieval altyapısına hazır mıyız?

Python çıktısı

5 belge okunuyor, boş doküman kontrolü yapılıyor.

Chunk üretimi

Boyut ve overlap testleri geçiyor.

Kaynak bağlama

Her chunk için doc_id / chunk_id korunuyor.

Embedding ve Chroma henüz bu pakette uygulanmıyor.